

交通与路线规划模块改造记录

1. 文档目的

本文记录 HelloAgents 智能旅行助手中交通与路线规划模块的改造过程，重点说明：

- 为什么原有地图路线不是真实道路轨迹；
- 如何在前端绘制真实的步行、驾车和公交路线；
- 如何在后端解析步行与驾车路线；
- 为什么公共交通 MCP 调用会一直阻塞；
- 为什么最终选择直接调用高德 REST API；
- 如何验证路线接口；
- 当前模块已经完成的范围和后续可以继续优化的内容。

本文既是本次开发记录，也可作为后续扩展交通模块时的学习资料。

2. 改造前的状态

项目原本已经具备以下基础能力：

- 景点拥有高德真实 POI ID、地址和经纬度；
- 酒店拥有高德真实 POI ID、地址和经纬度；
- `AmapService` 已配置高德 MCP 工具；
- 后端存在 `POST /api/map/route` 接口；
- 前端结果页能够在地图上显示景点 Marker。

但路线模块还没有形成真实数据闭环。

2.1 后端返回空对象

原来的 `AmapService.plan_route()` 虽然调用了路线工具，但没有解析返回结果：

```
# TODO: 解析实际的路线数据
return {}
```

这会产生两个问题：

1. `/api/map/route` 表面上可能返回“路线规划成功”，实际上没有有效数据；
2. 行程规划无法使用真实距离和耗时进行校验。

2.2 前端只绘制景点连线

原来的前端直接把景点坐标传给 `AMap.Polyline`：

```
const path = dayAttractions.map(attraction => [
  attraction.location.longitude,
```

```
attraction.location.latitude  
])
```

这种方式只会生成景点之间的直线，不会沿着道路、步行通道、公交线路或地铁线路绘制，因此它只能表示“访问顺序”，不能表示真实交通路线。

3. 模块目标和职责划分

本次改造采用以下职责划分：

```
规划 Agent  
└─ 决定每天访问哪些地点以及访问顺序  
  
后端路线服务  
├─ 获取真实距离  
├─ 获取真实耗时  
├─ 生成路线文字说明  
└─ 处理外部服务异常  
  
前端高德 JS API  
├─ 查询适合展示的路线  
└─ 在地图上绘制真实轨迹
```

Agent 不负责生成真实距离、耗时或轨迹，因为这些数据必须来自地图服务并且可以验证。

本次没有新增交通 Agent。对于路线查询这类确定性任务，直接调用地图服务比新增 Agent 更稳定、更快速，也更节省模型调用成本。

4. 前端真实路线绘制

4.1 加载高德路线插件

项目使用 `@amap/amap-jsapi-loader` 加载高德地图 JS API 2.0。

为了绘制真实路线，需要加载以下插件：

```
plugins: [  
  'AMap.Marker',  
  'AMap.Polyline',  
  'AMap.InfoWindow',  
  'AMap.Walking',  
  'AMap.Driving',  
  'AMap.Transfer'  
]
```

对应关系：

用户交通方式	高德 JS 插件
步行	<code>AMap.Walking</code>
自驾、驾车	<code>AMap.Driving</code>
公共交通、公交、地铁	<code>AMap.Transfer</code>

4.2 设置 JS API 安全密钥

高德 JS API 的地图底图能够显示，不代表路线服务一定能正常调用。路线插件还需要正确配置 JS API 安全密钥：

```
;(window as any)._AMapSecurityConfig = {
  securityJsCode: import.meta.env.VITE_AMAP_SECURITY_SECRET
}
```

该配置必须在 `AMapLoader.load()` 之前执行。

如果没有配置，可能出现：

```
INVALID_USER_SCODE
USERKEY_PLAT_NOMATCH
```

注意：这里使用的是高德控制台中的 JS API 安全密钥，不是后端 Web 服务 Key。

4.3 按天分组

路线不能把全部旅行日期的景点串成一条线，因此需要先按 `dayIndex` 分组：

```
const dayGroups: Record<number, any[]> = {}

attractions.forEach(attraction => {
  if (!dayGroups[attraction.dayIndex]) {
    dayGroups[attraction.dayIndex] = []
  }

  dayGroups[attraction.dayIndex].push(attraction)
})
```

每一天再按照规划 Agent 给出的景点顺序查询相邻地点路线：

```
景点 1 → 景点 2 → 景点 3
```

4.4 根据交通方式选择插件

前端增加统一分发入口：

```
if (transportation.includes('步行')) {  
  return drawWalkingSegment(...)  
}  
  
if (  
  transportation.includes('驾车') ||  
  transportation.includes('自驾')  
) {  
  return drawDrivingSegment(...)  
}  
  
if (  
  transportation.includes('公共交通') ||  
  transportation.includes('公交') ||  
  transportation.includes('地铁')  
) {  
  return drawTransitSegment(...)  
}
```

曾经出现过一个问题：

```
transportation.includes('公交')
```

不能匹配“公共交通”，因为“公”和“交”在“公共交通”中并不连续。因此需要分别匹配“公共交通”和“公交”。

4.5 自动绘制真实轨迹

高德路线对象在创建时传入地图实例：

```
const walking = new AMap.Walking({  
  map,  
  autoFitView: false  
})
```

查询成功后，高德会自动把真实道路轨迹添加到地图，无需手动解析 `polyline`。

设置：

```
autoFitView: false
```

可以避免多段路线查询时地图视野反复跳动。

5. 后端步行与驾车路线解析

5.1 MCP 实际返回结构

步行和驾车工具返回的主要结构为：

```
route
├─ origin
├─ destination
└─ paths[]
    ├─ distance
    ├─ duration
    └─ steps[]
        ├─ instruction
        ├─ road
        ├─ distance
        ├─ orientation
        └─ duration
```

其中距离和耗时虽然代表数字，但实际返回的是字符串：

```
{
  "distance": "1279",
  "duration": "1023"
}
```

因此解析时需要显式转换：

```
distance = float(distance_value)
duration = int(duration_value)
```

5.2 解析顺序

解析器必须按照由外到内的顺序校验：

```
data 是否为 dict
→ route 是否为 dict
→ paths 是否为非空 list
→ 第一条 path 是否为 dict
→ distance 和 duration 是否存在且能转换
→ steps 是否为 list
```

正确的核心顺序是：

```
data = extract_json_value(raw_result)

if not isinstance(data, dict):
    raise ValueError("路线响应不是JSON对象")

route = data.get("route")

if not isinstance(route, dict):
    raise ValueError("路线响应缺少route对象")

paths = route.get("paths")

if not isinstance(paths, list) or not paths:
    raise ValueError("路线响应没有可用路径")
```

开发过程中曾把 `paths` 的校验写在赋值之前，导致：

```
UnboundLocalError: cannot access local variable 'paths'
```

这说明即使逻辑内容正确，变量赋值与校验的顺序也必须严格检查。

5.3 路线说明

步行和驾车的路线说明来自：

```
path["steps"][*]["instruction"]
```

将每个步骤使用中文分号连接后，写入 `RouteInfo.description`。

6. 公共交通 MCP 阻塞问题

6.1 现象

调用公共交通 MCP 工具时，终端停在：

```
使用 Stdio 传输 (命令): uvx amap-mcp-server
连接到 MCP 服务器...
连接成功!
```

之后既不返回结果，也不主动超时。

地址版和坐标版公共交通工具都会出现相同问题，因此可以排除单纯的地址地理编码问题。

6.2 绕过 MCP 进行对照实验

直接调用高德公交 REST API：

```
https://restapi.amap.com/v3/direction/transit/integrated
```

返回结果为：

```
HTTP 200
status = 1
info = OK
infocode = 10000
公交方案数量 = 4
```

这个结果证明：

- 高德 Web 服务 Key 正常；
- 公交接口权限正常；
- 请求参数正常；
- 当前网络可以访问高德接口；
- 故障发生在 MCP Server 或 MCP stdio 通信层。

6.3 根因分析

在 `amap-mcp-server` 的公共交通工具实现中，高德返回后存在：

```
data = response.json()
print(data)
```

项目使用的是 MCP stdio 传输。在 stdio 模式中，标准输出应当用于传输 MCP/JSON-RPC 协议消息。

公交响应体非常大，通常包含：

- 多套公交方案；
- 多个换乘段；
- 公交和地铁线路；
- 上下车站点；
- 步行步骤；
- 途经站点。

将整份 `data` 直接打印到标准输出，可能污染 JSON-RPC 消息流，或者阻塞 stdio 管道，最终导致客户端一直等待合法的 MCP 响应。

此外，该工具内部的 `requests.get()` 没有设置超时；HelloAgents 当前的 MCP 调用也没有外层超时。因此任意环节不结束时，调用可能永久等待。

6.4 为什么步行和驾车正常

步行和驾车的工具实现没有触发相同的大型公交响应打印问题，返回结构也更简单，因此能够通过 MCP 正常完成。

6.5 MCP 与直接 REST 的区别

直接 REST：

业务代码 → requests → 高德 API → JSON

MCP：

业务代码 → HelloAgents MCP 客户端 → uvx 子进程
→ amap-mcp-server → 高德 API
→ MCP 数据转换 → stdio JSON-RPC → 业务代码

MCP 不是另一套地图数据源，它内部仍然调用高德 REST API，只是增加了工具协议、进程通信和响应转换层。

MCP 更适合 Agent 动态发现和选择工具；对于业务代码中已经明确知道要调用哪个接口的场景，直接 REST 通常更简单、更可控。

7. 公交最终解决方案

本项目最终采用：

步行 → 高德 MCP
驾车 → 高德 MCP
公共交通 → 高德 REST API

7.1 公交调用流程

由于现有路线接口输入的是地址，而高德公交接口需要坐标，所以流程为：

起点地址 → 高德地理编码 REST → 起点坐标
终点地址 → 高德地理编码 REST → 终点坐标
起终点坐标 → 高德公交 REST → 公交方案
公交方案 → RouteInfo

7.2 请求超时

地理编码和公交请求都设置：

```
timeout=(5, 20)
```

含义：

- 建立连接最多等待 5 秒；
- 接收响应最多等待 20 秒。

这样即使外部接口异常，后端也不会无限等待。

7.3 公交响应结构

公交主要结构为：

```
route
├─ origin
├─ destination
├─ distance
├─ transits[]
│   ├─ duration
│   ├─ walking_distance
│   └─ segments[]
│       ├─ walking
│       ├─ bus.buslines[]
│       ├─ entrance
│       ├─ exit
│       └─ railway
```

因此公交必须使用独立的：

```
parse_transit_route_result()
```

不能复用处理 `route.paths` 的步行、驾车解析器。

7.4 当前公交方案选择策略

当前选择：

```
first_transit = transits[0]
```

也就是使用高德返回的第一套默认推荐方案。

后续可以根据以下指标自行排序：

- 总耗时最短；
- 总步行距离最少；
- 换乘次数最少；
- 费用最低；
- 是否包含地铁。

8. API 验证过程

8.1 Swagger 是什么

FastAPI 会自动生成 Swagger UI。启动后端后访问：

```
http://localhost:8000/docs
```

可以直接查看并调用项目 API。

8.2 Postman 与 Swagger

两者都能验证接口：

- Swagger：由 FastAPI 自动生成，适合快速调试；
- Postman：独立 API 调试工具，适合保存请求和测试不同环境。

8.3 ECONNREFUSED

曾经在 Postman 中出现：

```
ECONNREFUSED
```

它表示 `localhost:8000` 没有程序监听，通常是后端尚未启动，不代表路线业务代码错误。

后端启动命令：

```
cd backend
python -m uvicorn app.api.main:app --reload --host 127.0.0.1 --port 8000
```

8.4 公交接口验证结果

通过 `POST /api/map/route` 查询公交路线后，成功得到：

```
{
  "success": true,
  "message": "路线规划成功",
  "data": {
    "distance": 19192.0,
    "duration": 6245,
    "route_type": "transit",
    "description": "步行659米；乘坐103路；步行316米；乘坐地铁4号线大兴线；步行1756米"
  }
}
```

其中：

- 距离约 19.2 公里；
- 耗时约 1 小时 44 分钟；
- 路线说明由步行段和公交、地铁线路组合生成。

这证明以下链路已经打通：

```
POST /api/map/route
→ 地址地理编码
→ 高德公交 REST API
→ route.transits 解析
→ RouteInfo
→ FastAPI JSON 响应
```

9. 常见错误与排查方式

9.1 AMap.Walking is not a constructor

原因：没有加载 AMap.Walking 插件。

检查 AMapLoader.load() 的 plugins。

9.2 地图显示但路线不显示

依次检查：

1. 浏览器 Console 是否有路线查询错误；
2. 是否配置 `_AMapSecurityConfig`；
3. 当前交通方式是否匹配分发条件；
4. 高德回调状态是否为 `complete`；
5. 公交查询是否返回 `no_data`。

9.3 MCP 停在“连接成功”

说明 MCP Server 已启动，但工具调用没有返回。应检查：

- MCP 工具内部网络请求；
- 是否有向 stdout 打印非协议内容；
- 是否缺少超时；
- 能否绕过 MCP 直接调用原始接口。

9.4 ECONNREFUSED

检查后端是否启动，以及请求端口是否与 Uvicorn 监听端口一致。

9.5 422 Unprocessable Entity

表示请求 JSON 缺少字段、字段类型错误，或者值没有通过 Pydantic 校验。

9.6 500 Internal Server Error

表示请求已经进入后端，但业务代码、解析器或外部服务调用发生异常。应同时检查 API 响应和后端终端堆栈。

10. 当前完成范围

已经完成：

- 前端步行真实轨迹；
- 前端驾车真实轨迹；
- 前端公交真实轨迹；
- 后端步行 MCP 查询和解析；
- 后端驾车 MCP 查询和解析；
- 后端公交 REST 查询和解析；
- 公交地理编码；
- 外部 REST 请求超时；
- `/api/map/route` 返回真实距离、耗时和说明；
- 外部服务失败时不再返回虚假的空成功数据。

当前模块可以视为完成了独立路线查询和地图展示的 MVP。

11. 后续优化清单

以下内容暂不属于当前 MVP，可在后续迭代中完成。

11.1 使用现有 POI 坐标

当前公交接口为了兼容地址请求，会进行两次地理编码，整个请求耗时约 8 秒。

景点和酒店已经拥有真实坐标，接入旅行计划时应直接传坐标：

```
POI 坐标 → 公交 REST
```

这样可以减少两次外部请求。

11.2 将路线写入 DayPlan

未来可以新增：

```
routes: list[RouteLeg]
```

并生成完整路线链：

```
酒店 → 景点 1 → 景点 2 → 景点 3 → 酒店
```

11.3 日程可行性校验

计算：

```
当天总活动时间
= 景点游览时间
+ 所有交通时间
```

如果超过合理阈值，例如 10 小时，可以提示用户减少景点。

11.4 公交多方案排序

不再固定选择第一套方案，而是根据用户偏好选择：

- 少步行；
- 少换乘；
- 最快；
- 最省钱。

11.5 真实交通预算

- 步行费用为 0；
- 公交费用仅使用接口返回的可靠价格；
- 驾车费用需要结合过路费、里程和业务规则；
- 没有可靠数据时使用 **None**，不要让模型猜测。

11.6 完善错误语义

可以进一步区分：

- 参数错误：HTTP 422；
- 外部地图服务失败：HTTP 503；
- 地图响应无法解析：HTTP 502；
- 未查询到路线：根据业务设计返回 404 或明确业务错误。

11.7 MCP 懒加载

当前创建 **AmapService** 时仍会初始化 MCP。即使只查询公交，也会进行一次 MCP 工具发现。

未来可以改成按需初始化：

```
调用步行或驾车时才初始化 MCP
调用公交时直接使用 REST
```

12. 本次改造的核心经验

1. 地图上的连线不等于真实路线，真实轨迹应由专业路线服务计算。

2. Agent 适合决策和生成建议，不适合编造距离、耗时和坐标。
3. 外部服务返回值必须先确认真实结构，再编写解析器。
4. 步行、驾车和公交的响应结构不一定相同，应分别建模。
5. MCP 是工具协议层，不是数据源；MCP 失败时可以绕过它验证原始 API。
6. stdio 协议服务不能随意向 stdout 打印调试数据。
7. 所有外部网络请求都应该设置超时。
8. 不要吞掉异常并返回空对象，否则容易产生“假成功”。
9. 先完成独立接口闭环，再接入复杂的旅行计划主流程。
10. 前端展示和后端业务校验可以分工，不必强制使用同一条技术链路。